

Main Application Page — Detailed Context

The **Main Application Page** is the core workspace of the system. It has two purposes:

1. **Generate Mode** — generate a new 3D reconstruction from drone data.
2. **View Mode** — visualize an already-generated/premade 3D model.

The page should keep the **3D visualization as the main focus**, with controls and input panels appearing around it rather than occupying the entire screen.

1. Initial State

After the user clicks **Enter Application** from the landing page:

- Main Application Page opens.
 - User is presented with two clear options:
 - **Generate Model**
 - **View Model**
 - No reconstruction or ML processing starts automatically.
 - The user chooses the workflow they want.
-

2. Generate Model

User sees →

A centered upload/generation panel containing:

Required inputs

- Drone Video
- GPS Coordinates
- Flight Metadata

Optional inputs

- IMU Data
- Barometric Altitude

- Camera Intrinsic Parameters
- RTK/PPK Corrections

Primary action:

Generate 3D Model

User flow →

Upload → Validate → Generate → Processing

After clicking Generate:

- Files are sent to the **backend**.
- Backend creates a reconstruction job.
- Backend passes the required data to the **ML model**.
- ML pipeline performs reconstruction.
- Backend manages generated assets, metadata and job status.
- Frontend displays the processing state.

Loading / Processing UI →

The user should see:

- Overall progress
- Current processing stage
- Processing status
- Estimated/elapsed processing time
- Visual loading animation

Stages can correspond to the actual pipeline, for example:

Data Preparation → Keyframe Selection → 3D Reconstruction → Scene Understanding → Georeferencing → Asset Generation → Complete

The frontend should **not fake completion**. The UI should be ready to consume real backend job-status updates later.

3. After Generation

When the backend reports the reconstruction as complete:

Default → Cesium View

The application automatically opens the **Cesium View**.

The user can then switch between:

- **Cesium**
- **Semantic**
- **Point Cloud**

All three views represent the same reconstructed survey.

Switching views does **not** trigger reconstruction again.

4. View Model

View Mode is purely for **visualization/inspection of an existing model**.

User sees →

A model upload panel asking for an existing/premade model.

Supported formats:

- GLB
- GLTF
- OBJ + MTL + Textures
- LAS
- PLY
- FBX

For the initial frontend prototype:

GLB is the primary supported visualization asset.

Example:

Upload GLB → Load Model → 3D/Cesium View

Important distinction

View Mode does **not**:

- Run the ML reconstruction pipeline.
- Require drone video/GPS/flight metadata.
- Generate a new reconstruction.

It simply loads an existing model for visualization.

5. Main Visualization Workspace

Once a model is available, the application becomes a **full-screen 3D workspace**.

Primary navigation

A persistent view switcher provides:

Cesium | Semantic | Point Cloud

Cesium

- Default view after generation.
- Survey model at its real geographic location.
- Optional global/world context.
- Area measurement and geographic inspection.

Semantic

- Classified 3D scene.
- Buildings, roads, terrain, vegetation, trees, forest, water, etc.
- Vegetation can additionally be differentiated by **observed visual color**.
- Semantic filtering and object inspection.

Point Cloud

- Dense georeferenced point cloud.
 - Point selection and technical geometric inspection.
 - Point-level information and measurements.
-

6. Common Application Controls

Controls that are relevant across views should remain consistent:

- View switcher
- Layer visibility
- Selection
- Measurement
- Clear selection
- Reset camera/view
- Model information
- Coordinates/location information
- Full-screen visualization

View-specific controls should appear only when relevant.

7. Cross-View Interaction

The three views must behave as different representations of the **same spatial dataset**.

Common identifiers:

- Point ID
- Object ID
- Geographic XYZ
- Latitude/Longitude
- Semantic class

Example:

Cesium area selection →

- Corresponding points highlighted in Point Cloud.
- Corresponding terrain/structures highlighted in Semantic View.

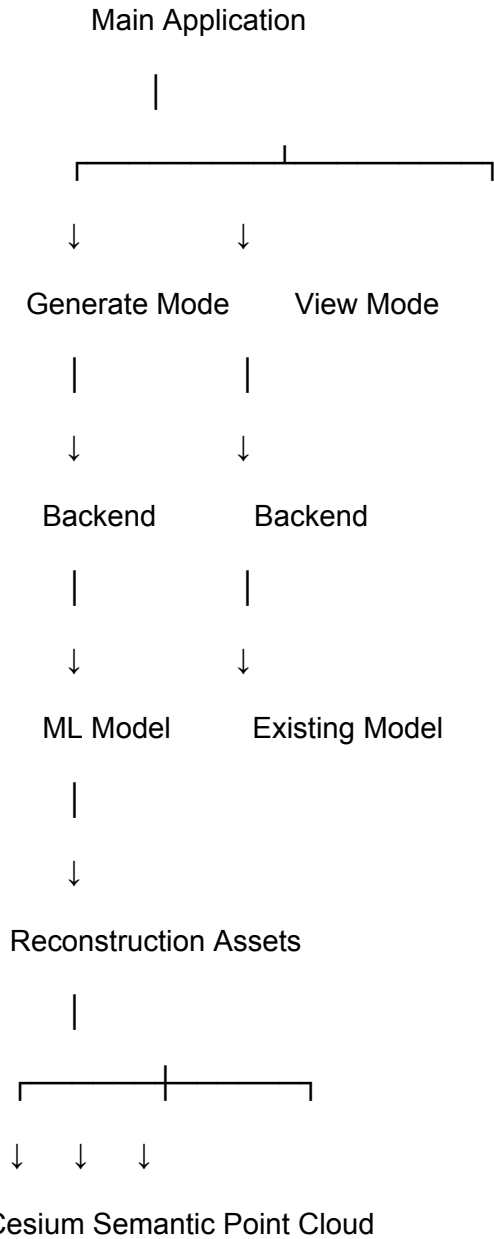
Point Cloud selection →

- Corresponding semantic objects highlighted.
- Corresponding geographic location shown in Cesium.

This synchronization should be handled through the backend-generated spatial/object mapping, not by running ML again.

8. Frontend ↔ Backend ↔ ML Architecture

The frontend should have a clean separation:



Frontend

Responsible for:

- User interaction
- File uploads
- Job progress display

- Visualization
- Measurements/selection UI
- View switching

Backend

Responsible for:

- API
- File handling
- Job management
- ML orchestration
- Asset management
- Metadata
- Spatial/object mappings

ML Model

Responsible for:

- Reconstruction
 - Camera/geometry estimation
 - Point cloud generation
 - Semantic understanding
 - Terrain/object reconstruction
 - Georeferencing-related processing as defined by the pipeline
-

9. Prototype vs Final System

Now — Prototype

Use premade assets:

- Premade GLB
- Premade point cloud
- Premade semantic data where available

The objective is to finalize:

- UI
- UX
- View switching

- Controls
- Measurements
- Selection
- Cross-view interaction

Later — Final System

Replace the premade assets with:

Backend → ML Model → Generated Assets → Frontend

The frontend architecture should remain essentially the same.

10. Core Design Principle

The Main Application should feel like **one continuous geospatial 3D workspace**, not three separate pages.

The user's journey is:

Landing → Main Application → Generate/View → Model → Cesium (default) → Semantic / Point Cloud → Inspect / Measure / Analyze

And the fundamental distinction is:

Generate Mode creates the reconstruction. View Mode visualizes an existing reconstruction. The three visualization modes are different views of the same model/data.